

# WashU-2 CPU

## Week 1 Report

Digital Logic & VHDL First Steps

---

|             |                                                  |
|-------------|--------------------------------------------------|
| Author      | Ayush Pati                                       |
| Roll No.    | 25B3901                                          |
| Email       | 25b3901@iitb.ac.in                               |
| Institution | IIT Bombay — Electrical Engineering, Second Year |
| Programme   | SOC 2026 — WashU-2 CPU Mentorship                |
| Submitted   | Week 1 · June 2026                               |

## Exercise A — 3-Input AND Gate

---

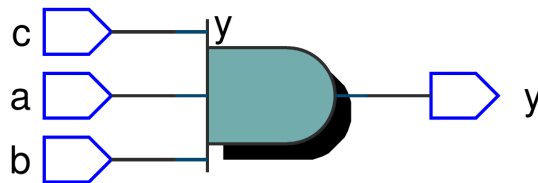
**Entity:** and3\_gate

A 3-input AND gate built using structural VHDL. The entity *and3\_gate* accepts three `std_logic` inputs (**a**, **b**, **c**) and produces a single output **y**. The architecture chains two 2-input AND gates via an internal signal, demonstrating component instantiation and concurrent signal assignment.

### RTL Netlist View

---

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



### Simulation Waveform

---

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



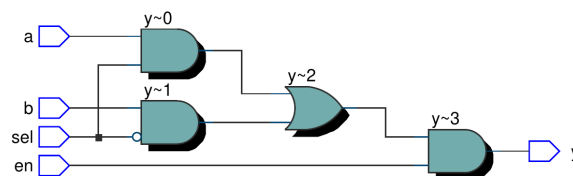
## Exercise B — 2:1 Multiplexer with Enable

**Entity:** mux2\_en

A 2-to-1 multiplexer extended with an active-high enable signal. When **en** = '1' and **sel** = '0', output **y** follows input **a**; when **sel** = '1', **y** follows **b**. When **en** = '0' the output is forced low regardless of the select line. Implemented using Boolean logic with AND/OR/NOT gates in structural style.

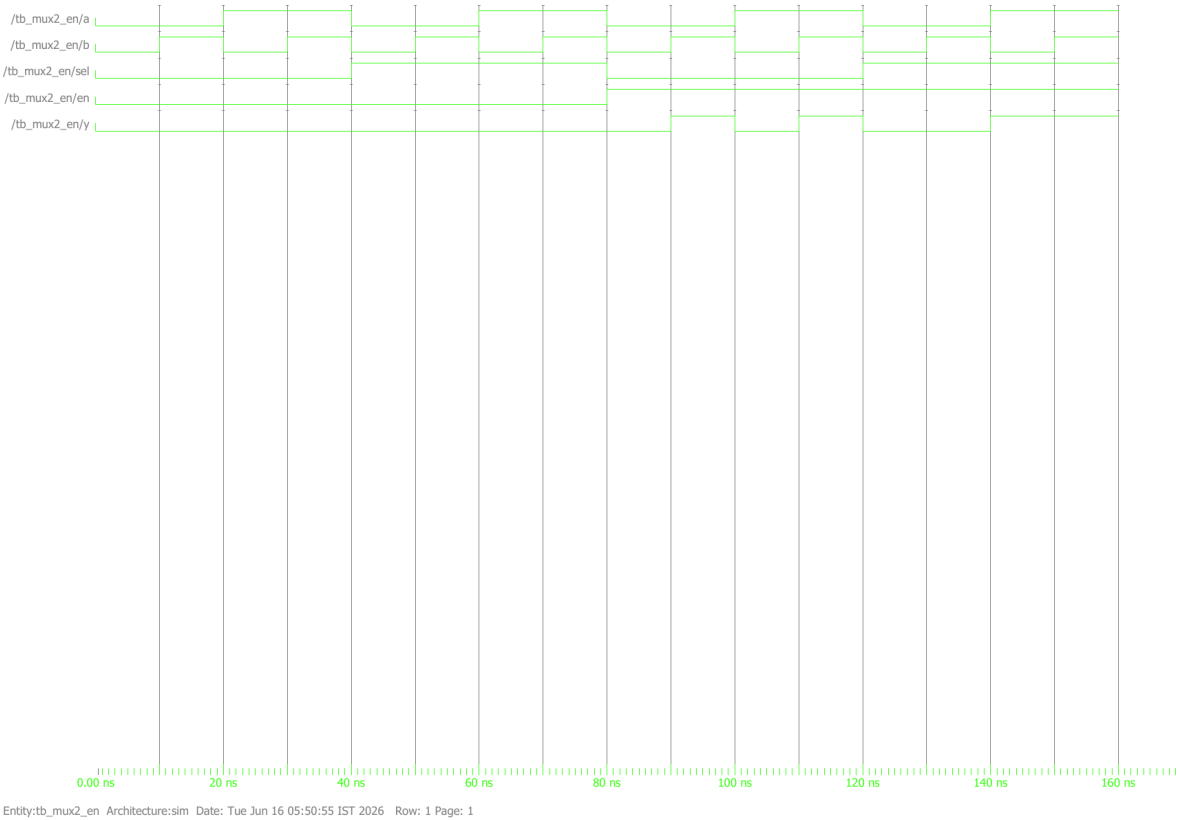
### RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



### Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



## Exercise 1 — XOR Gate from Structural Gates

---

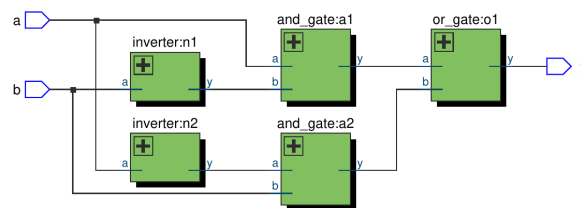
**Entity:** xor\_structural

An XOR gate built entirely from AND, OR, and NOT primitives — no behavioural xor operator used. The identity  $y = (a \text{ AND } \text{NOT } b) \text{ OR } (\text{NOT } a \text{ AND } b)$  is realised using four gate instantiations and two internal signals. This exercise demonstrates structural composition and the port map syntax for connecting sub-components.

### RTL Netlist View

---

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



### Simulation Waveform

---

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



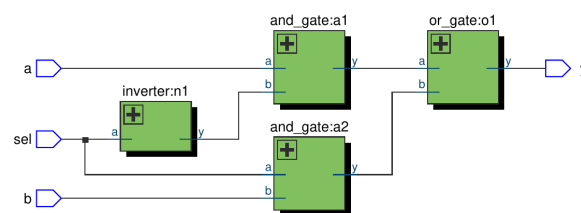
## Exercise 2 — 2:1 Multiplexer

**Entity:** mux2

A 2-to-1 multiplexer using AND/OR/NOT gates in structural VHDL. Implements  $y = (a \text{ AND } \text{NOT } sel) \text{ OR } (b \text{ AND } sel)$ . The provided testbench cycles through all four input combinations for both select values, verifying correct routing of inputs to the output.

### RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



### Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.





## Bugs & Debugging Notes

---

Both bugs encountered this week stemmed from VHDL syntax unfamiliarity — a common experience when transitioning from software to hardware description languages.

### Bug 1 — Semicolons vs. Commas in Port Declarations

Port entries were separated with commas instead of semicolons, as is common in most programming languages. For example:

```
a : in std_logic, -- WRONG
b : in std_logic,
y : out std_logic
```

**Fixed:**

```
a : in std_logic; -- CORRECT
b : in std_logic;
y : out std_logic -- no semicolon on the last port
```

VHDL uses semicolons to separate port entries, and the final port must have **no** trailing semicolon. Quartus flagged both errors with line numbers, making them easy to locate. Re-reading the entity syntax in Module 2 resolved it immediately.

### Bug 2 — Missing 'map' Keyword in Component Instantiation

When instantiating sub-components, the keyword `map` was omitted after `port`:

```
g1 : entity work.and_gate
port (a => a, b => b, y => w); -- WRONG
```

**Fixed:**

```
g1 : entity work.and_gate
port map (a => a, b => b, y => w); -- CORRECT
```

VHDL requires the exact two-word phrase `port map` for component instantiation. The error message pointed to the opening parenthesis, which was initially confusing. Referring back to the Module 4 instantiation example revealed the missing keyword.